

个人报告：人脸识别与红外温度活体检测考勤系统

一、个人工作概述

在本项目中，我主要负责将原有的人脸识别考勤系统扩展为“人脸识别 + 红外温度检测 + 眨眼活体检测”的综合考勤系统。项目整体涉及昇腾开发板、USB 摄像头、STM32F103、MLX90640 红外热成像传感器、Windows 串口转发脚本、Flask Web 后端、SQLite 数据库以及前端页面交互。

我的主要工作不是单独完成某一个模块，而是围绕整个系统的实际运行过程进行调试、修改、联调和完善，使系统最终能够完成从用户注册、人脸识别、红外温度判断、眨眼活体检测到自动考勤记录写入的完整流程。

二、主要完成内容

1. 完成昇腾开发板端人脸识别系统调试

项目初期，系统已经具备基础的人脸识别框架，但在实际运行时存在摄像头打不开、网页访问异常、数据库无法写入等问题。我首先对昇腾开发板端的运行环境和代码结构进行了排查。

我完成的工作包括：

1. 检查 Flask 服务启动状态；
2. 确认开发板 IP 地址和访问端口；
3. 修改 Flask 服务监听方式，使其能够通过局域网访问；
4. 检查摄像头设备节点；
5. 通过 OpenCV 遍历 `/dev/video0` 到 `/dev/video3`；
6. 确认实际可用摄像头设备为 `/dev/video2`；
7. 修改摄像头初始化逻辑，使系统能够正常读取摄像头画面。

通过这些工作，解决了网页打不开、摄像头无法打开、视频流不显示等问题，为后续人脸识别和活体检测打下了基础。

2. 完成用户管理功能修复

在用户管理模块中，系统最初存在用户列表显示异常、删除用户后编号不连续、注册用户后识别不稳定等问题。

我完成的工作包括：

1. 修改用户列表接口，使前端能够正常显示用户；
2. 处理数据库中人脸特征字段导致 JSON 返回异常的问题；
3. 将数据库真实 ID 和页面显示序号分离；
4. 删除用户后，页面显示编号能够重新从 1 开始排序；
5. 优化用户注册流程，使上传照片或摄像头抓拍注册后能够保存用户信息和人脸特征；
6. 检查注册照片质量对后续识别结果的影响。

通过修改后，用户管理页面可以正常完成用户添加、显示、删除和编号重排，满足系统测试和演示需求。

3. 完成数据库与考勤记录功能完善

在测试过程中，数据库曾出现只读错误和考勤记录字段缺失问题。我对数据库结构和权限进行了排查，并补充了考勤记录管理功能。

我完成的工作包括：

1. 解决 SQLite 数据库只读问题；
2. 检查数据库文件和目录权限；
3. 为考勤记录补充温度字段；
4. 增加考勤记录查询接口；
5. 增加考勤记录删除接口；
6. 在考勤页面加入删除考勤记录功能；
7. 方便后续反复测试同一用户的识别和活体检测流程。

通过该部分工作，系统不仅能够自动写入考勤记录，还可以删除测试记录，便于调试和验收。

4. 完成 MLX90640 红外温度数据接收流程

项目中使用 STM32F103 读取 MLX90640 红外热成像传感器，并通过串口输出 32×24 的温度矩阵。为了让昇腾开发板能够使用这些温度数据，我完成了 Windows 端温度转发和昇腾端温度接收的联调。

我完成的工作包括：

1. 分析 STM32 串口输出的温度矩阵格式；
2. 修改 Windows 端 `send\_temp\_to\_ascend.py` 脚本；
3. 解决单行多个温度值无法转换的问题；
4. 实现每行 32 个温度点的解析；
5. 累计 24 行形成完整 768 点温度帧；
6. 统计每帧温度的 `min`、`avg`、`max`、`p90` 等指标；
7. 将完整温度帧通过 HTTP POST 发送到昇腾板；
8. 在 Flask 中增加 `/api/temperature` 接口；
9. 在昇腾板端缓存最新红外温度矩阵。

最终实现了从 MLX90640 到 STM32，再到 Windows 脚本，最后到昇腾板 Flask 服务的完整温度数据链路。

5. 完成人脸 ROI 红外温度判断

仅仅接收整张红外温度矩阵并不能直接判断活体，因为环境中可能存在其他热源。因此，我将红外温度判断从“全局温度判断”改为“人脸 ROI 温度判断”。

我完成的工作包括：

1. 在人脸识别成功后获取当前人脸框；
2. 将摄像头画面中的人脸框映射到 32×24 红外矩阵；
3. 根据映射点选取局部红外 ROI；
4. 计算 ROI 区域内的高位温度、环境温度、温差和热点数量；

5. 根据这些指标判断该区域是否符合人体热区特征；
6. 避免使用整张红外矩阵直接判断活体。

该方案提高了红外温度判断的针对性，减少了环境热源对系统的干扰。

6. 完成眨眼活体检测功能

为了防止普通照片直接通过人脸识别和温度判断，我在系统中加入了眨眼动作检测。眨眼检测作为最终活体条件之一，只有用户完成眨眼动作后，系统才允许写入考勤记录。

我完成的工作包括：

1. 在人脸识别成功后截取人脸区域；
2. 取人脸上半部分作为眼睛检测区域；
3. 使用 OpenCV 眼睛检测方法判断眼睛状态变化；
4. 设计眨眼检测流程；
5. 将眨眼检测接入自动考勤流程；
6. 确保只有在红外 ROI 温度通过后才进入眨眼检测；
7. 避免单纯照片或普通热源直接完成考勤。

经过多次调试后，系统能够识别眨眼动作，并将其作为活体检测的重要依据。

7. 优化摄像头视频流卡顿问题

在项目测试过程中，摄像头画面曾出现卡顿甚至卡死的问题。经过分析，主要原因是摄像头采集、模型推理、人脸识别和活体检测都集中在同一流程中，导致视频帧读取被阻塞。

我完成的优化包括：

1. 将摄像头采集和识别处理分离；
2. 采集线程持续读取最新画面；
3. 识别线程按一定间隔处理最新帧；
4. 视频流接口只负责输出缓存帧；
5. 降低视频编码质量，减少传输压力；

6. 控制视频流输出帧率，减少 CPU 占用。

优化后，摄像头画面流畅度明显提升，系统能够更稳定地进行人脸识别和活体检测。

8. 完成重复考勤控制与测试便利功能

为了避免同一用户站在摄像头前被连续多次打卡，我加入了重复考勤控制逻辑。同时，为了方便测试，又增加了考勤记录删除功能。

我完成的工作包括：

1. 增加同一用户的考勤冷却时间；
2. 防止短时间内重复写入多条考勤记录；
3. 增加考勤记录删除接口；
4. 在前端考勤页面增加删除功能；
5. 删除考勤记录后可以再次测试同一用户。

这样既保证了实际使用时不会频繁重复打卡，也方便在开发和验收阶段进行反复测试。

9. 完成项目代码整理与 GitHub 上传

在项目功能基本完成后，我将开发板上的工程文件导出到本地电脑，并整理上传到 GitHub 仓库。

我完成的工作包括：

1. 在昇腾板上将 `case1` 工程打包；
2. 使用 MobaXterm 或 SCP 将压缩包下载到 Windows 电脑；
3. 解压并整理工程目录；
4. 初始化 Git 仓库；
5. 配置 Git 用户名和邮箱；
6. 处理 GitHub HTTPS 推送失败问题；
7. 配置 SSH key；
8. 使用 SSH 成功连接 GitHub；
9. 处理 GitHub 100MB 文件限制；

10. 排除过大的模型文件;
11. 添加 `.gitignore` 和 `.gitattributes`;
12. 将工程上传到 GitHub 仓库;
13. 将 `case1/` 和 `MLX90640/` 分别作为独立文件夹上传, 避免文件散落在仓库根目录。

最终形成了较清晰的仓库结构, 便于后续查看、下载和部署。

10. 完成项目文档编写

为了便于项目展示、验收和后续复现, 我整理并编写了多份 Markdown 文档。

我完成的文档包括:

1. 项目设计报告;
2. 最终项目报告;
3. GitHub 仓库使用说明;
4. 系统验收报告;
5. 带实物连接图的验收报告;
6. 本个人工作报告。

文档内容涵盖项目背景、系统架构、硬件组成、软件模块、数据流程、红外温度判断方案、眨眼活体检测方案、功能验收结果、创新点、常见问题、运行步骤和后续优化方向。

三、项目中的关键问题与解决方法

1. 摄像头无法打开

****问题:**** 原代码默认打开 `/dev/video0`, 但实际可用摄像头是 `/dev/video2`。

****解决方法:**** 通过 OpenCV 遍历 `/dev/video0` 到 `/dev/video3`, 确认 `/dev/video2` 可以正常读取图像, 并修改摄像头初始化逻辑。

2. 数据库只读

****问题：**** 运行程序时出现：

```
sqlite3.OperationalError: attempt to write a readonly database
```

****解决方法：**** 检查数据库文件和目录权限，调整文件写权限，使程序能够正常写入用户和考勤记录。

3. 温度数据解析失败

****问题：**** STM32 串口输出一行包含多个温度值，原脚本按单个浮点数解析，导致转换失败。

****解决方法：**** 修改 Windows 端脚本，使其支持每行 32 个温度点，累计 24 行后组成完整 768 点温度帧。

4. 温度数据传到昇腾板后无法用于活体判断

****问题：**** 温度数据虽然发送成功，但摄像头识别线程无法正确读取 Flask 接收到的温度缓存。

****解决方法：**** 统一温度缓存读取方式，确保摄像头识别线程使用的是 Flask 当前进程中最新的温度矩阵。

5. 静态照片和普通热源可能误判

****问题：**** 如果只依赖人脸识别或简单温度阈值，照片和普通热源可能造成误判。

****解决方法：**** 将活体判断改为：

```
人脸识别成功 + 红外人脸 ROI 温度通过 + 眨眼动作检测通过
```

三个条件同时满足才允许考勤。

四、个人收获

通过本项目，我对嵌入式 AI 系统和多传感器融合应用有了更完整的理解。

主要收获包括：

- 1. 熟悉了昇腾开发板上运行 Python、Flask 和模型推理程序的流程；
- 2. 掌握了 USB 摄像头在 Linux 系统中的调试方法；
- 3. 理解了 OpenCV 摄像头读取、视频流输出和图像处理的基本方法；
- 4. 掌握了 SQLite 数据库在 Web 项目中的基本使用；
- 5. 熟悉了 STM32F103 与 MLX90640 红外传感器的数据采集流程；
- 6. 掌握了串口数据解析和 HTTP 数据转发方法；
- 7. 理解了红外温度矩阵与摄像头人脸 ROI 对应的基本思路；
- 8. 学会了通过多条件组合提升活体检测安全性；
- 9. 掌握了 GitHub 仓库整理、SSH 推送和大文件处理方法；
- 10. 提升了系统联调、问题定位和工程文档编写能力。

五、个人总结

本项目从最初的普通人脸识别系统，逐步扩展为融合红外温度判断和眨眼活体检测的考勤系统。整个过程中，我完成了摄像头调试、用户管理修复、数据库完善、红外温度数据链路搭建、人脸 ROI 温度判断、眨眼活体检测、考勤记录管理、代码上传和文档编写等工作。

最终系统能够完成：

用户注册 → 摄像头识别人脸 → 红外 ROI 温度判断 → 眨眼活体检测 → 自动考勤

的完整闭环。

通过本项目，我不仅完成了一个可以实际运行的系统，也进一步理解了 AI 推理、嵌入式传感器、Web 后端、数据库和前端页面之间的协同关系。该项目具有较强的综合性，对提升实际工程开发能力和系统调试能力有明显帮助。